ADGSTUDIOS

DisputeGuard

Production-Grade Card Dispute Management Platform

Software Engineering Specification

Full-Stack Banking Technology Demonstration
Rust · Next.js · PostgreSQL · Redis
Ollama · Agentic AI · Docker · Kubernetes

Organisation:	ADGSTUDIOS (Pty) Ltd
System:	ADGSTUDIOS DisputeGuard
Document Type:	Engineering / Architecture Specification
Version:	1.0
Date:	October 2026

A secure, auditable and scalable demonstration platform for transaction dispute management in a financial-services environment.

Contents

1 Executive Summary **4**

2 Project Objectives **5**

3 Scope **5**
3.1 Required Scope 5
3.2 Extended Scope 5

4 System Actors **6**

5 High-Level Architecture **7**

6 Frontend Architecture **7**
6.1 Customer Workspace 7
6.2 Analyst Workspace 8

7 Backend Architecture **8**

8 Proposed Repository Structure **9**

9 Domain Model **9**

10 Database Design **10**
10.1 Important Relationships 10

11 Financial Data Representation **10**

12 Dispute State Machine **11**

13 Duplicate Dispute Prevention **12**

14 Authentication and Authorisation **12**

15 Role-Based Access Control **12**

16 Idempotency **13**

17 Optimistic Concurrency **13**

18 Audit Model **14**

19 Mathematical Risk Engine **14**

20 Amount Normalisation **14**

21 Statistical Transaction Anomaly **15**

22 Robust Statistics **15**

23 Transaction Velocity **16**

24 Geographical Analysis **16**

25 Risk Interpretation **17**

26 AI Architecture **17**

27 Structured AI Output **18**

28 Bounded Agentic Investigation Loop **18**

29 Agent Safety Bounds **19**

30 AI Tool Allowlist **19**

31 AI Explainability **20**

32 Redis Design **20**

33 Rate Limiting **21**

34 API Design **21**

35 Dispute Creation Transaction **22**

36 Security Threat Model **22**

37 Prompt Injection Defence **23**

38 Evidence Upload Security **23**

39 Observability **24**

40 Metrics **24**

41 Latency Objectives **24**

42 Caching **25**

43 Docker Deployment **25**

44 Container Health **25**

45 Kubernetes Deployment **26**

46 Kubernetes Architecture **27**

47 Horizontal Scaling **27**

48 Horizontal Pod Autoscaling **28**

49 Failure Behaviour **28**

49.1 Redis Failure 28

49.2 Ollama Failure 28

49.3 Worker Failure 28

50 Testing Strategy **29**

51 Critical Test Cases **29**

52 Concurrency Test **29**

53 Property-Based Testing **30**

54 Performance Testing **30**

55 CI/CD **30**

56 Configuration and Secrets **31**

57 Example Customer Journey **31**

58 Example AI Investigation **32**

59 Example Analyst Interface **33**

60 Dispute Timeline **33**

61 Production Engineering Principles **34**

62 Engineering Trade-Offs **34**
- 62.1 Why Rust? . . . 34
- 62.2 Why PostgreSQL? . . . 35
- 62.3 Why Redis? . . . 35
- 62.4 Why Ollama? . . . 35
- 62.5 Why Kubernetes? . . . 35

63 Definition of Done **35**

64 Presentation Strategy **36**

65 Final Architecture Principle **37**

66 Conclusion **37**

1 Executive Summary

ADGSTUDIOS DisputeGuard is a production-oriented card dispute management platform designed to demonstrate the engineering of a secure financial-services workflow.

The system allows customers to:

- authenticate securely;
- view mock card transactions;
- select eligible transactions;
- submit transaction disputes;
- provide dispute reasons and supporting evidence;
- receive a unique dispute reference;
- monitor the dispute lifecycle through a timeline interface.

Authorised dispute analysts can:

- review incoming disputes;
- inspect transaction information;
- inspect mathematical risk indicators;
- review customer evidence;
- request additional information;
- transition disputes through controlled states;
- use AI-assisted case summarisation;
- approve or reject cases according to authorised workflow rules;
- inspect a complete audit trail.

The core system is intentionally designed so that deterministic business logic remains independent of generative artificial intelligence.

The design principle is:

$$\boxed{\text{Mathematics Computes} \rightarrow \text{Software Enforces} \rightarrow \text{AI Assists} \rightarrow \text{Humans Decide}}$$

The proposed technology stack consists of:

- Next.js and TypeScript for the user interfaces;
- Rust, Axum and Tokio for the backend API;
- PostgreSQL as the authoritative transactional datastore;
- Redis for caching, rate limiting and asynchronous coordination;
- Ollama for locally controlled LLM inference;
- an agentic bounded investigation loop for AI assistance;
- Docker for reproducible execution;
- Kubernetes as an optional production-style deployment model.

2 Project Objectives

The primary objectives are:

1. Implement the required card-dispute workflow completely.
2. Demonstrate production-quality software engineering.
3. Protect transaction and dispute information through explicit authorisation rules.
4. Prevent duplicate and invalid dispute operations.
5. Maintain a traceable dispute history.
6. Demonstrate mathematical risk analysis.
7. Demonstrate safe AI augmentation without delegating financial decisions to an LLM.
8. Provide automated testing and reproducible deployment.
9. Demonstrate horizontal scalability without making Kubernetes necessary for local execution.

3 Scope

3.1 Required Scope

The minimum functional system shall provide:

1. mock transaction retrieval;
2. transaction presentation;
3. dispute submission;
4. persistent dispute storage;
5. dispute status management;
6. dispute timeline visualisation.

3.2 Extended Scope

DisputeGuard extends the minimum implementation with:

- authentication;
- role-based access control;
- customer ownership validation;
- evidence handling;
- analyst workflows;
- audit logging;
- deterministic risk scoring;

- statistical anomaly indicators;
- AI-assisted classification;
- AI-assisted summarisation;
- bounded agent execution;
- Redis-backed asynchronous jobs;
- structured logging;
- metrics;
- Docker Compose;
- Kubernetes manifests;
- CI/CD support.

4 System Actors

The primary actors are:

Actor	Responsibilities
Customer	Views their own transactions, creates disputes, uploads supporting evidence and monitors case progress.
Dispute Analyst	Reviews assigned or queued disputes, examines risk signals, requests evidence, records notes and performs authorised transitions.
Administrator	Manages administrative configuration and authorised users.
AI Worker	Performs bounded classification, summarisation and investigation-assist operations. It cannot make final financial decisions.
System	Enforces authentication, authorisation, state transitions, persistence, auditability, rate limits and integrity constraints.

5 High-Level Architecture

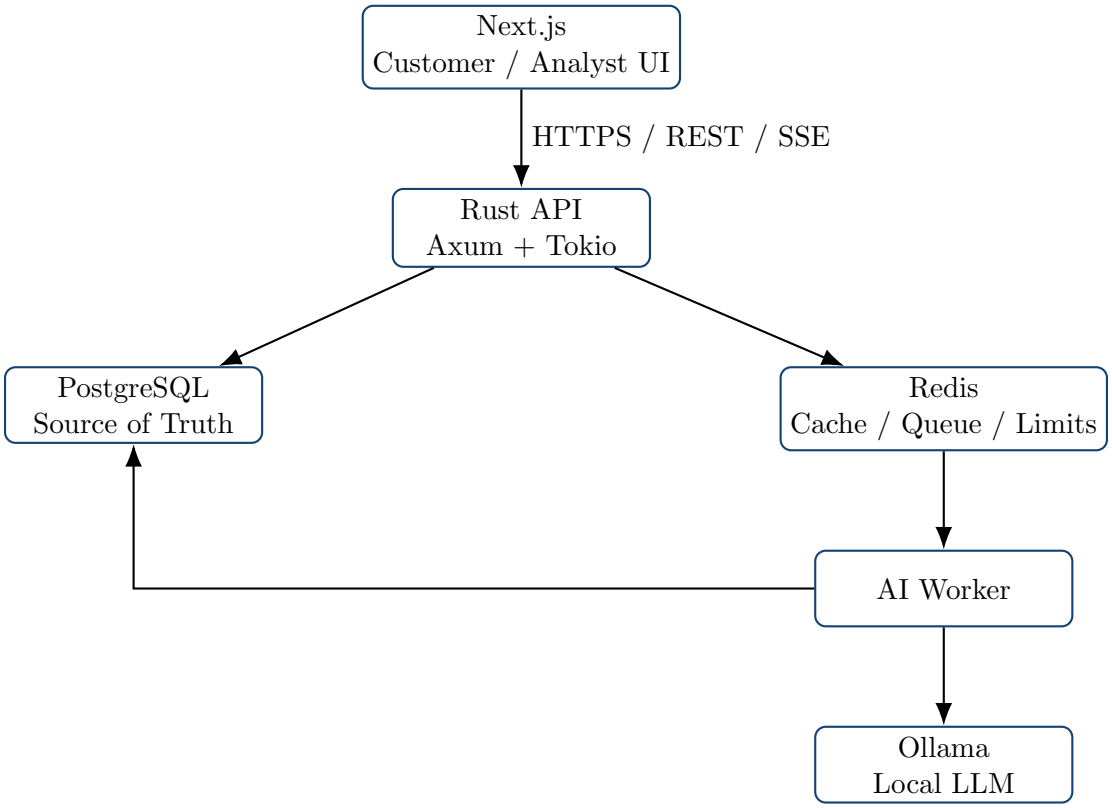


Figure 1: High-level DisputeGuard architecture.

6 Frontend Architecture

The frontend shall be implemented using Next.js and TypeScript.

The application contains two primary workspaces.

6.1 Customer Workspace

The customer interface provides:

- login;
- dashboard;
- transaction history;
- transaction details;
- dispute eligibility;
- dispute creation wizard;
- evidence submission;
- dispute history;
- status timeline.

6.2 Analyst Workspace

The analyst interface provides:

- case queue;
- filters;
- priority ordering;
- case details;
- transaction details;
- risk analysis;
- evidence viewer;
- analyst notes;
- AI case summary;
- controlled state transitions;
- audit event viewer.

7 Backend Architecture

The backend shall be written in Rust.

Recommended libraries include:

- Axum for HTTP routing;
- Tokio for asynchronous execution;
- SQLx for PostgreSQL access;
- Serde for serialisation;
- Tower middleware;
- Tracing for structured telemetry.

The backend should use a layered architecture:

$$\text{HTTP} \rightarrow \text{Application} \rightarrow \text{Domain} \rightarrow \text{Infrastructure}.$$

The domain layer shall not depend directly upon HTTP, Redis or PostgreSQL implementation details.

8 Proposed Repository Structure

```
ADG-DisputeGuard/
|
|-- apps/
|   |-- web/
|   |-- api/
|   `-- ai-worker/
|
|-- crates/
|   |-- domain/
|   |-- application/
|   |-- persistence/
|   `-- common/
|
|-- database/
|   |-- migrations/
|   `-- seeds/
|
|-- infrastructure/
|   |-- docker/
|   `-- kubernetes/
|
|-- docs/
|   |-- specification.tex
|   |-- threat-model.md
|   `-- diagrams/
|
|-- tests/
|
|-- docker-compose.yml
|-- Dockerfile
|-- README.md
`-- .github/workflows/
```

9 Domain Model

Let:

$$\mathcal{U} = \text{set of users},$$

$$\mathcal{C} = \text{set of cards},$$

$$\mathcal{T} = \text{set of transactions},$$

and

$$\mathcal{D} = \text{set of disputes}.$$

A transaction is represented by:

$$t = (id, customer, card, merchant, amount, currency, timestamp, status).$$

A dispute is represented by:

$$d = (id, transaction, customer, reason, status, createdAt, updatedAt).$$

10 Database Design

Core tables include:

- users;
- cards;
- transactions;
- disputes;
- dispute_events;
- dispute_evidence;
- analyst_notes;
- audit_logs;
- ai_analysis;

10.1 Important Relationships

$$User \rightarrow Card$$

$$Card \rightarrow Transaction$$

$$Transaction \rightarrow Dispute$$

$$Dispute \rightarrow DisputeEvent$$

$$Dispute \rightarrow Evidence$$

$$Dispute \rightarrow AIAnalysis.$$

11 Financial Data Representation

Floating-point arithmetic shall not be used for authoritative monetary values.

For currency with two decimal places, an amount may be represented in minor units:

$$R1234.56 \rightarrow 123456.$$

Therefore:

$$amount_{\text{minor}} \in \mathbb{Z}.$$

Alternatively, a fixed precision PostgreSQL numeric type may be used.
This avoids binary floating-point representation errors such as:

$$0.1 + 0.2 \neq 0.3$$

under standard binary floating-point arithmetic.

12 Dispute State Machine

Let the state space be:

$$S = \left\{ \begin{array}{l} \texttt{SUBMITTED},\ \texttt{UNDER_REVIEW},\ \texttt{EVIDENCE_REQUIRED}, \\ \texttt{EVIDENCE_RECEIVED},\ \texttt{INVESTIGATING},\ \texttt{APPROVED}, \\ \texttt{REJECTED},\ \texttt{REFUND_PENDING},\ \texttt{RESOLVED} \end{array} \right\}.$$

Let:

$$E = \text{set of authorised dispute events.}$$

Define the transition function:

$$T : S \times E \to S \cup \{\bot\}.$$

Here,

$$\bot$$

denotes an invalid transition.
Examples include:

$$T(SUBMITTED, BEGIN_REVIEW) = UNDER_REVIEW,$$

$$T(UNDER_REVIEW, REQUEST_EVIDENCE) = EVIDENCE_REQUIRED,$$

$$T(EVIDENCE_REQUIRED, EVIDENCE_SUBMITTED) = EVIDENCE_RECEIVED,$$

$$T(INVESTIGATING, APPROVE) = APPROVED,$$

and

$$T(INVESTIGATING, REJECT) = REJECTED.$$

An invalid transition is represented by:

$$T(SUBMITTED, ISSUE_REFUND) = \bot.$$

The backend shall reject any transition for which:

$$T(s, e) = \bot.$$

13 Duplicate Dispute Prevention

Define the set of active dispute states as:

$$S_A \subseteq S.$$

The system invariant is:

$$\forall t \in \mathcal{T}, \quad |\{d \in \mathcal{D} : d.transaction = t \wedge d.status \in S_A\}| \leq 1.$$

This invariant shall be enforced primarily by PostgreSQL rather than by Redis.

The API may perform an early existence check for user experience, but database-level enforcement remains authoritative.

14 Authentication and Authorisation

Define:

$$Owner(u, t)$$

to mean user u owns the account/card associated with transaction t.

Then:

$$CanViewTransaction(u, t) \iff Owner(u, t) \vee Role(u) \in \{ANALYST, ADMIN\}.$$

For customer dispute creation:

$$CanDispute(u, t) \Rightarrow Owner(u, t).$$

An identifier supplied by the browser shall never establish ownership.

The backend shall independently verify:

$$transaction.customer_id = authenticated_user.id.$$

15 Role-Based Access Control

Define roles:

$$R = \{CUSTOMER, ANALYST, ADMIN\}.$$

Permissions shall be explicit.

For example:

$$P(CUSTOMER) = \left\{ \begin{array}{l} \texttt{VIEW_OWN_TRANSACTION}, \\ \texttt{CREATE_OWN_DISPUTE}, \\ \texttt{VIEW_OWN_DISPUTE} \end{array} \right\}.$$

$$P(ANALYST) = P(CUSTOMER) \cup \left\{ \begin{array}{l} \texttt{VIEW_CASE}, \texttt{REVIEW_CASE}, \\ \texttt{REQUEST_EVIDENCE}, \texttt{TRANSITION_CASE} \end{array} \right\}.$$

Administrative privileges shall not implicitly be exposed through frontend controls alone; they shall be verified server-side.

16 Idempotency

Financial APIs must tolerate retries.

For dispute creation, the client may supply:

$$K = \text{Idempotency-Key}.$$

The server stores:

$$(K, u, h, r),$$

where:

- K is the key;
- u is the authenticated user;
- h is a hash of the request;
- r is the original result.

If the same request is retried:

$$(K, u, h) = (K', u', h'),$$

the original result is returned.

If the key is reused with a different payload:

$$K = K' \land h \neq h',$$

the request shall be rejected.

17 Optimistic Concurrency

A dispute shall contain a version:

$$v_d \in \mathbb{N}.$$

An update is permitted only if:

$$v_{\text{request}} = v_{\text{database}}.$$

After successful mutation:

$$v_{new} = v_{old} + 1.$$

Conceptually:

```
UPDATE disputes
SET status = $1,
    version = version + 1
WHERE id = $2
  AND version = $3;
```

If zero rows are updated, a concurrent modification has occurred.

18 Audit Model

Every security-sensitive or case-sensitive operation shall produce an audit event.

An audit event may be represented as:

$$a = (actor, action, resource, timestamp, metadata).$$

For state changes, store:

$$(s_{old}, s_{new}, actor, timestamp).$$

The audit history shall be append-oriented.

Application users shall not be permitted to silently rewrite historical audit events.

19 Mathematical Risk Engine

The mathematical risk engine provides deterministic signals for analyst triage.

It does **not** determine whether fraud occurred.

For dispute d, define:

$$R(d) = w_A A(d) + w_V V(d) + w_M M(d) + w_H H(d) + w_G G(d),$$

where:

$$w_i \geq 0$$

and

$$\sum_i w_i = 1.$$

The components represent:

$$\begin{aligned} A(d) &= \text{amount anomaly}, \\ V(d) &= \text{transaction velocity anomaly}, \\ M(d) &= \text{merchant risk signal}, \\ H(d) &= \text{historical behaviour anomaly}, \\ G(d) &= \text{geographical anomaly}. \end{aligned}$$

Therefore:

$$0 \leq R(d) \leq 1$$

provided every component lies in $[0, 1]$.

20 Amount Normalisation

Raw monetary values may have highly skewed distributions.

A logarithmic transformation may therefore be used:

$$A(x) = \min\left(1, \frac{\ln(1+x)}{\ln(1+x_{\max})}\right).$$

This reduces domination by extremely large values.

21 Statistical Transaction Anomaly

For customer u, suppose historical transaction amounts are:

$$x_1, x_2, \ldots, x_n.$$

The sample mean is:

$$\bar{x}_u = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

The sample variance is:

$$s_u^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x}_u)^2.$$

Therefore:

$$s_u = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x}_u)^2}.$$

For new transaction x:

$$z_x = \frac{x - \bar{x}_u}{s_u}.$$

The magnitude:

$$|z_x|$$

provides a simple indication of how unusual the transaction amount is relative to the customer's historical behaviour.

For demonstration purposes:

$$|z_x| > 3$$

may be labelled highly unusual.

This is an anomaly indicator, not proof of fraud.

22 Robust Statistics

Transaction distributions are frequently non-normal and contain outliers.

A robust alternative uses the median:

$$m = median(x_1, \ldots, x_n)$$

and Median Absolute Deviation:

$$MAD = median(|x_i - m|).$$

A modified z-score can be defined as:

$$z_i^* = \frac{0.6745(x_i - m)}{MAD}.$$

This is less sensitive to extreme historical transactions than the ordinary mean and standard deviation.

23 Transaction Velocity

Let:

$$N_u(t - \Delta t, t)$$

represent the number of transactions made by customer u during window $(t - \Delta t, t]$.
Define transaction velocity:

$$V_u(t) = \frac{N_u(t - \Delta t, t)}{\Delta t}.$$

Suppose historical velocity has mean:

$$\mu_{V,u}$$

and standard deviation:

$$\sigma_{V,u}.$$

Then:

$$z_V = \frac{V_u(t) - \mu_{V,u}}{\sigma_{V,u}}.$$

Large positive values indicate unusually high transaction frequency.

24 Geographical Analysis

For two geographic points:

$$(\phi_1, \lambda_1)$$

and

$$(\phi_2, \lambda_2),$$

the Haversine formula gives:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right).$$

Then:

$$c = 2\arctan 2\left(\sqrt{a}, \sqrt{1-a}\right).$$

Distance is:

$$d = R_E c,$$

where approximately:

$$R_E = 6371 \text{ km}.$$

If the elapsed time is:

$$\Delta t,$$

the implied travel velocity is:

$$v_{required} = \frac{d}{\Delta t}.$$

An implausibly large value may contribute to a geographical anomaly signal.

However, merchant location, online payments, VPN usage, delayed settlement and travel must prevent this signal from being interpreted as proof of fraud.

25 Risk Interpretation

For presentation purposes only, risk scores may be divided into bands:

$$0 \leq R < 0.30 \Rightarrow LOW,$$

$$0.30 \leq R < 0.60 \Rightarrow MODERATE,$$

$$0.60 \leq R < 0.80 \Rightarrow HIGH,$$

$$0.80 \leq R \leq 1 \Rightarrow VERY\ HIGH.$$

These thresholds shall be configuration values rather than hard-coded financial policy.

26 AI Architecture

Ollama shall provide local model inference.

Permitted AI use cases include:

- dispute classification;
- customer-description summarisation;
- analyst case summarisation;
- evidence recommendations;
- investigation assistance.

AI shall not:

- approve a dispute;

- reject a dispute;
- issue a refund;
- change balances;
- bypass RBAC;
- access arbitrary customer information;
- execute unrestricted database queries.

The central invariant is:

$$AIRecommendation(d) \not\Rightarrow FinalDecision(d).$$

Instead:

$$FinalDecision(d) \Rightarrow AuthorisedHumanAction(d).$$

27 Structured AI Output

AI responses shall use structured schemas wherever possible.

Example:

```
{
  "suggested_category": "DUPLICATE_TRANSACTION",
  "confidence": 0.91,
  "summary": "Customer reports duplicate billing.",
  "priority": "NORMAL",
  "recommended_evidence": [
    "Merchant receipt",
    "Order confirmation"
  ]
}
```

The backend shall validate model output before persistence.

Invalid output shall be discarded or marked as failed analysis.

28 Bounded Agentic Investigation Loop

The optional investigation agent follows a bounded iterative pattern.

Let the agent state at iteration k be:

$$A_k = (C_k, O_k, B_k),$$

where:

$$C_k = \text{current case context},$$

$$O_k = \text{observations obtained through authorised tools},$$

and

$$B_k = \text{remaining execution budget.}$$

The next state is:

$$A_{k+1} = F(A_k, \tau_k),$$

where:

$$\tau_k \in \mathcal{T}_{allowed}$$

is an approved tool call.
The loop terminates when:

$$Complete(A_k) = true,$$

or when any resource constraint is reached.

29 Agent Safety Bounds

Let:

$$k_{\max}$$

be the maximum number of reasoning iterations.
Then:

$$k \leq k_{\max}.$$

Similarly:

$$N_{tools} \leq N_{tools}^{max},$$

$$N_{tokens} \leq N_{tokens}^{max},$$

and

$$T_{execution} \leq T_{max}.$$

Therefore termination is guaranteed by bounded execution even if the model repeatedly requests additional actions.

A possible demonstration configuration is:

$$k_{\max} = 8.$$

30 AI Tool Allowlist

Possible read-only tools include:

- get_transaction;
- get_customer_transaction_summary;

- get_dispute_timeline;
- get_evidence_metadata;
- get_risk_signals;
- get_merchant_summary.

The agent shall not receive a generic SQL execution tool.
Formally:

$$\tau_k \in \mathcal{T}_{allowed}.$$

If:

$$\tau_k \notin \mathcal{T}_{allowed},$$

execution shall be rejected.

31 AI Explainability

The interface shall not attempt to expose private model chain-of-thought.
Instead, the system records structured execution events such as:

```
{
  "step": 3,
  "action": "get_risk_signals",
  "result": "high_amount_anomaly",
  "next": "request_manual_review"
}
```

The analyst receives:

- deterministic mathematical signals;
- tool execution history;
- concise AI-generated summary;
- AI confidence metadata where applicable.

32 Redis Design

Redis shall not be the source of truth for dispute records.
Potential responsibilities include:

- rate limiting;
- cached read models;
- temporary session state where appropriate;
- background job queues;
- AI job coordination;

- short-lived tool-result caching.

The persistence hierarchy is:

$$\boxed{PostgreSQL = Authoritative}$$

$$\boxed{Redis = Ephemeral/Performance/Coordination}$$

33 Rate Limiting

A token bucket may be used.
Let:

$$B(t)$$

be the number of available tokens at time t.
Let:

$$r$$

be the refill rate and C the maximum capacity.
Then:

$$B(t + \Delta t) = \min\left(C, B(t) + r\Delta t\right).$$

A request consumes one token:

$$B' = B - 1.$$

If:

$$B < 1,$$

the request is rate limited.

34 API Design

Example endpoints include:

```
POST   /api/v1/auth/login
POST   /api/v1/auth/logout

GET    /api/v1/transactions
GET    /api/v1/transactions/:id

POST   /api/v1/disputes
GET    /api/v1/disputes
GET    /api/v1/disputes/:id
GET    /api/v1/disputes/:id/timeline

POST   /api/v1/disputes/:id/evidence

GET    /api/v1/analyst/disputes
```

```
GET    /api/v1/analyst/disputes/:id

POST   /api/v1/analyst/disputes/:id/transition
POST   /api/v1/analyst/disputes/:id/notes

POST   /api/v1/analyst/disputes/:id/ai-analysis

GET    /health
GET    /ready
```

35 Dispute Creation Transaction

Dispute creation shall occur within a database transaction.

Conceptually:

```
BEGIN;

SELECT transaction
FOR UPDATE;

VERIFY ownership;
VERIFY eligibility;
VERIFY no active dispute;

INSERT dispute;
INSERT dispute_event;
INSERT audit_log;

COMMIT;
```

If any invariant fails:

$$ROLLBACK.$$

This ensures:

$$Atomicity \land Consistency.$$

36 Security Threat Model

The system shall consider at least:

- broken access control;
- IDOR;
- SQL injection;
- cross-site scripting;
- CSRF where relevant;
- credential attacks;
- brute force;

- malicious evidence uploads;
- privilege escalation;
- replayed requests;
- duplicate submissions;
- race conditions;
- sensitive-data leakage;
- LLM prompt injection;
- malicious model output;
- excessive agent tool execution.

37 Prompt Injection Defence

Uploaded evidence and customer descriptions are untrusted input.

Therefore:

$$CustomerContent \neq SystemInstruction.$$

The agent shall treat customer-provided text as data.

Tool access shall be enforced by application code rather than by prompt instructions alone.

Hence:

$$Security \not\equiv \text{“}Please\ do\ not\ misuse\ tools.\text{”}$$

Instead:

$$Security = Authorisation + ToolAllowlist + SchemaValidation + ExecutionBounds.$$

38 Evidence Upload Security

Evidence handling shall include:

- file size limits;
- permitted MIME types;
- generated server-side filenames;
- extension validation;
- storage outside directly executable paths;
- authorisation checks;
- malware scanning where available;
- metadata persistence;
- audit events.

39 Observability

The system shall provide structured logs.

Each request should include a correlation identifier:

$$CID \in UUID.$$

Logs may include:

- timestamp;
- correlation ID;
- route;
- response status;
- latency;
- authenticated actor identifier;
- safe error classification.

Sensitive data shall not be unnecessarily logged.

40 Metrics

Useful metrics include:

$$RequestsPerSecond,$$

$$ErrorRate,$$

$$P_{50}, P_{95}, P_{99},$$

$$ActiveDisputes,$$

$$QueueDepth,$$

$$AIJobDuration,$$

and

$$DatabasePoolUtilisation.$$

41 Latency Objectives

For non-AI endpoints, an example engineering objective is:

$$P_{95}(T_{API}) < 200 \text{ ms}$$

under a documented benchmark workload.
Similarly:

$$P_{99}(T_{API}) < 500 \text{ ms}.$$

These are engineering targets rather than unsupported production claims.
AI operations shall be asynchronous where latency is significant.

42 Caching

For a cache:

$$H = \frac{N_{hit}}{N_{hit} + N_{miss}}$$

defines the cache hit ratio.
Caching may be used for:

- safe reference data;
- dashboard aggregates;
- temporary AI tool results.

Authoritative dispute state shall not depend solely on cached data.

43 Docker Deployment

The complete project shall provide a runnable Docker-based environment.
A local topology may contain:

```
services:
  web:
  api:
  worker:
  postgres:
  redis:
  ollama:
```

The expected developer workflow shall be approximately:

```
docker compose up --build
```

The README shall document all required environment variables and setup steps.

44 Container Health

The backend shall expose:

```
/health
/ready
```

The liveness endpoint answers:

Alive?

The readiness endpoint answers:

$$CanServeTraffic?$$

A service may therefore satisfy:

$$Alive = true$$

while:

$$Ready = false.$$

45 Kubernetes Deployment

Kubernetes is an optional production-style deployment and is not required to evaluate the application locally.

Suggested resources include:

- Namespace;
- ConfigMap;
- Secret references;
- API Deployment;
- Web Deployment;
- Worker Deployment;
- Services;
- Ingress;
- HorizontalPodAutoscaler;
- readiness probes;
- liveness probes.

46 Kubernetes Architecture

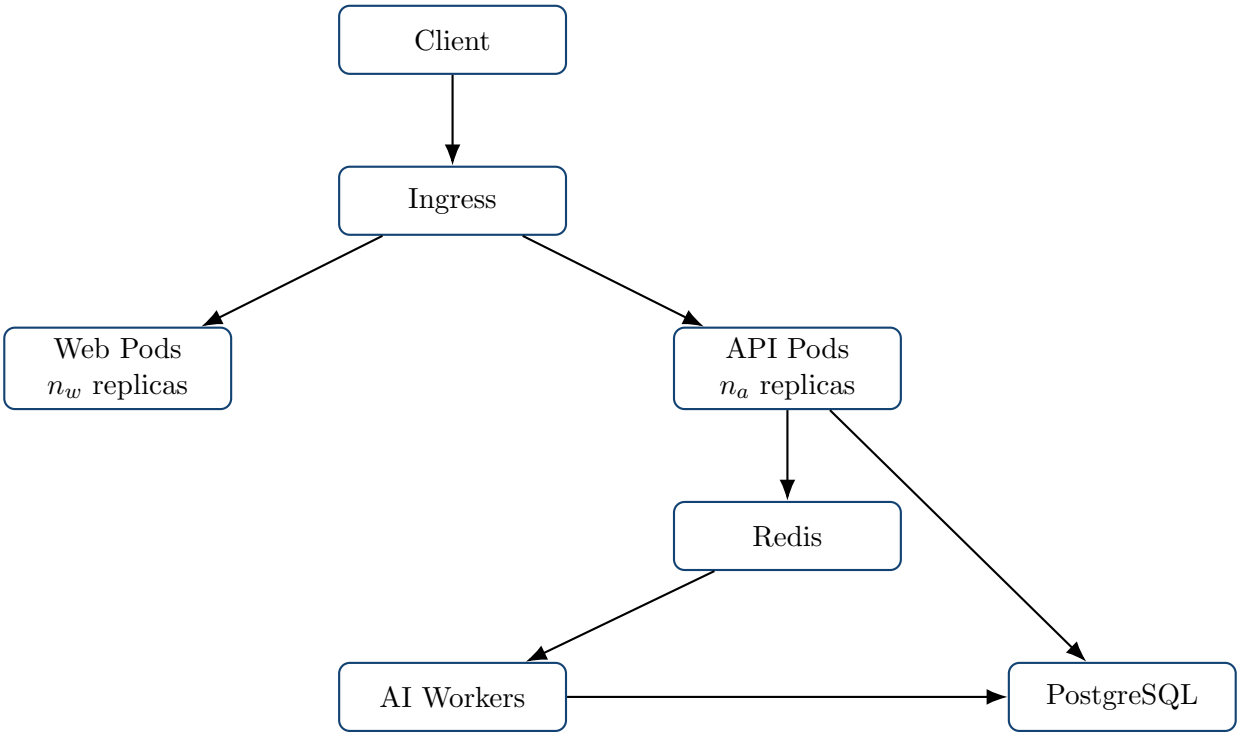


Figure 2: Illustrative Kubernetes deployment.

47 Horizontal Scaling

For n API replicas with individual effective capacities C_i:

$$C_{ideal} = \sum_{i=1}^{n} C_i.$$

Actual capacity satisfies approximately:

$$C_{actual} \leq \sum_{i=1}^{n} C_i$$

because of shared bottlenecks including:

- PostgreSQL;
- Redis;
- network bandwidth;
- connection pools;
- storage;
- AI inference capacity.

Therefore the system does not assume perfect linear scaling.

48 Horizontal Pod Autoscaling

Suppose:

$$U_c$$

is current average CPU utilisation and

$$U_t$$

is target utilisation.
A simplified desired replica estimate is:

$$N_{desired} = \left\lceil N_{current} \frac{U_c}{U_t} \right\rceil .$$

For example:

$$N_{current} = 3, \quad U_c = 80\%, \quad U_t = 60\%.$$

Then:

$$N_{desired} = \left\lceil 3 \frac{0.80}{0.60} \right\rceil = \lceil 4 \rceil = 4.$$

49 Failure Behaviour

49.1 Redis Failure

If Redis becomes unavailable:

- authoritative dispute records remain in PostgreSQL;
- non-essential caching may degrade;
- AI/background jobs may pause;
- rate-limiting behaviour must fail according to explicit policy.

49.2 Ollama Failure

If Ollama becomes unavailable:

$$CoreDisputeWorkflow = Available.$$

AI functionality shall degrade gracefully.

49.3 Worker Failure

A worker crash must not corrupt dispute state.
Jobs shall be retried only where operations are safe and idempotent.

50 Testing Strategy

Testing shall include:

1. unit tests;
2. integration tests;
3. API tests;
4. database tests;
5. frontend component tests;
6. end-to-end tests;
7. authorisation tests;
8. concurrency tests;
9. AI schema validation tests.

51 Critical Test Cases

The test suite shall verify at minimum:

- a customer can view their own transactions;
- a customer cannot view another customer's transaction;
- a customer can dispute an eligible transaction;
- duplicate active disputes are rejected;
- invalid state transitions are rejected;
- analyst-only endpoints reject customers;
- idempotent retries do not create duplicate disputes;
- concurrent dispute creation preserves uniqueness;
- AI failure does not break core dispute functionality;
- malformed AI output is rejected;
- unauthorised agent tools cannot execute.

52 Concurrency Test

Suppose m requests simultaneously attempt to dispute transaction t.

Let:

$$D_t = \{d \in \mathcal{D} : d.transaction = t \wedge d.status \in S_A\}.$$

After all requests complete, the invariant must remain:

$$|D_t| \leq 1.$$

This shall be verified through an automated concurrency test.

53 Property-Based Testing

The state machine is suitable for property-based testing.

For every generated state/event pair:

$$(s, e),$$

the implementation should satisfy:

$$Implementation(s, e) = T(s, e).$$

Furthermore, no generated sequence of valid application operations should violate:

$$ActiveDisputes(transaction) \leq 1.$$

54 Performance Testing

Benchmark scenarios should include:

- transaction list retrieval;
- transaction detail retrieval;
- dispute creation;
- dispute timeline retrieval;
- analyst queue retrieval.

Results shall report:

$$Throughput, \quad P_{50}, \quad P_{95}, \quad P_{99}, \quad ErrorRate.$$

Any benchmark claims shall include:

- hardware;
- dataset size;
- concurrent users;
- test duration;
- software configuration.

55 CI/CD

The repository should contain automated CI steps for:

1. Rust formatting;
2. Rust linting;
3. Rust tests;
4. frontend linting;

5. frontend tests;
6. production frontend build;
7. Docker image build;
8. optional vulnerability scanning.

A failing test shall prevent successful CI completion.

56 Configuration and Secrets

Secrets shall never be committed to Git.

The repository may provide:

```
.env.example
```

containing placeholder configuration.

Sensitive values include:

- database credentials;
- signing keys;
- API secrets;
- object storage credentials.

Kubernetes manifests shall reference secrets rather than embedding production values.

57 Example Customer Journey

Consider a customer with transaction:

$$x = R8750.00.$$

Suppose their historical mean is:

$$\bar{x} = R643.00$$

with standard deviation:

$$s = R920.00.$$

Then:

$$z = \frac{8750 - 643}{920}.$$

First calculate:

$$8750 - 643 = 8107.$$

Therefore:

$$z = \frac{8107}{920}.$$

Hence:

$$z \approx 8.81.$$

Thus:

$$|z| \approx 8.81 > 3.$$

The system may therefore display:

> Transaction amount is highly unusual relative to the customer's historical transaction amounts.

This does not establish fraud.
The customer then submits a dispute.
The backend verifies:

$$Owner(customer, transaction) = true.$$

It verifies:

$$ActiveDisputes(transaction) = 0.$$

It then atomically creates:

$$Dispute + DisputeEvent + AuditEvent.$$

The customer receives a reference such as:

```
DSP-2026-000184
```

58 Example AI Investigation

After dispute creation, an asynchronous job may start.
Iteration 0:

$$A_0 = (C_0, \emptyset, B_0).$$

The agent requests transaction information:

$$\tau_0 = get_transaction.$$

The result is added:

$$O_1 = O_0 \cup \{transaction\}.$$

The agent requests deterministic risk signals:

$$\tau_1 = get_risk_signals.$$

Then:

$$O_2 = O_1 \cup \{amount, velocity, geographic, history\}.$$

If enough information exists:

$$Complete(A_2) = true.$$

The model produces a structured summary.
No dispute decision is automatically performed.

59 Example Analyst Interface

The analyst may see:

```
CASE: DSP-2026-000184

Transaction
-----------
Merchant:       Example Electronics
Amount:         R8,750.00
Date:           30 October 2026

Risk Signals
------------
Amount anomaly:       HIGH
Velocity anomaly:     LOW
Geographic anomaly:   MODERATE
Historical anomaly:   HIGH

AI Assistance
-------------
Suggested category:
CARD_NOT_PRESENT / UNRECOGNISED

Recommendation:
MANUAL REVIEW

Case Status
-----------
UNDER_REVIEW
```

The analyst, not the AI system, performs the authorised workflow action.

60 Dispute Timeline

A typical timeline may be:

```
30 Oct 2026 14:32
* Dispute submitted

30 Oct 2026 14:32
* Dispute acknowledged

31 Oct 2026 09:14
* Review started

31 Oct 2026 09:17
* Evidence requested

```

```
01 Nov 2026 10:02
* Evidence received

01 Nov 2026 13:20
* Investigation started

02 Nov 2026 11:10
* Analyst decision recorded
```

Each timeline entry is derived from persisted dispute events rather than invented client-side state.

61 Production Engineering Principles

The implementation shall follow these principles:

1. PostgreSQL is authoritative.
2. Redis is disposable.
3. AI is optional.
4. AI does not determine financial outcomes.
5. Every protected resource is authorised server-side.
6. Critical invariants are enforced at multiple layers.
7. Monetary arithmetic is exact.
8. Retries are expected.
9. Concurrent requests are expected.
10. Every important state transition is auditable.
11. Services expose health and readiness.
12. Failure of optional components must degrade gracefully.
13. Local execution must remain simple.

62 Engineering Trade-Offs

62.1 Why Rust?

Rust is not selected solely because of raw benchmark performance.

The design benefits include:

- memory safety;
- strong compile-time guarantees;
- explicit error handling;
- predictable resource usage;
- high-performance asynchronous execution;
- strong type modelling for domain states.

62.2 Why PostgreSQL?

The dispute workflow requires strong transactional semantics and relational integrity.

PostgreSQL provides:

- ACID transactions;
- constraints;
- indexes;
- row locking;
- mature relational querying.

62.3 Why Redis?

Redis supports ephemeral performance and coordination workloads without replacing PostgreSQL as the authoritative database.

62.4 Why Ollama?

Ollama enables local model execution and allows AI functionality to be demonstrated without coupling core dispute processing to a remote LLM provider.

62.5 Why Kubernetes?

Kubernetes is included to demonstrate a possible scalable deployment model.

It is deliberately not required for local evaluation.

Docker Compose remains the simplest supported execution path.

63 Definition of Done

The project is complete when:

1. a fresh clone can be started according to the README;
2. Docker builds successfully;
3. seeded users can authenticate;
4. customers can view mock transactions;
5. customers can submit disputes;
6. duplicate disputes are prevented;
7. customers can view dispute timelines;
8. analysts can review disputes;
9. authorised state transitions work;
10. unauthorised transitions fail;
11. audit events are persisted;

12. deterministic risk indicators are visible;
13. optional AI analysis works when Ollama is available;
14. the system remains usable when AI is unavailable;
15. automated tests pass;
16. Kubernetes manifests are documented if included.

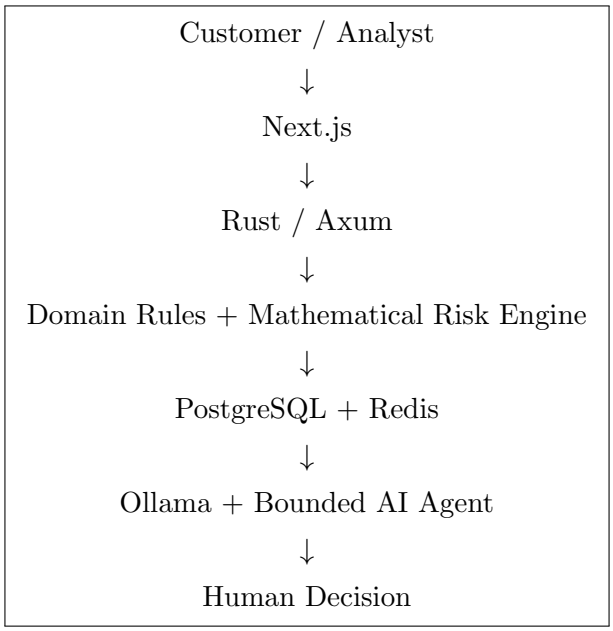
64 Presentation Strategy

The demonstration should proceed in the following order:

1. Present the business problem.
2. Present the architecture.
3. Login as a customer.
4. View transactions.
5. Open an unusual transaction.
6. Explain mathematical risk indicators.
7. Submit a dispute.
8. Demonstrate duplicate prevention.
9. Display the dispute timeline.
10. Login as an analyst.
11. Review the case.
12. Demonstrate AI-assisted analysis.
13. Demonstrate the bounded agent execution events.
14. Perform an authorised state transition.
15. Display the audit trail.
16. Explain PostgreSQL consistency.
17. Explain Redis responsibilities.
18. Demonstrate tests.
19. Demonstrate Docker.
20. Present the optional Kubernetes deployment.

65 Final Architecture Principle

The overall system can be summarised as:



with deployment provided through:

$$\boxed{Docker \rightarrow Docker\ Compose \rightarrow Optional\ Kubernetes}$$

The central design philosophy remains:

$$\boxed{Correctness > Complexity}$$

$$\boxed{Security > Convenience}$$

and

$$\boxed{AI\ Assists \quad Human\ Decides.}$$

66 Conclusion

ADGSTUDIOS DisputeGuard demonstrates how a relatively small card dispute brief can be engineered as a realistic financial-services software system.

The architecture deliberately separates deterministic domain rules, mathematical risk analysis, transactional persistence, ephemeral coordination and generative AI.

The result is intended to demonstrate not merely the ability to build a full-stack interface, but the ability to reason about:

- correctness;
- security;
- mathematics;
- concurrency;

- distributed systems;
- artificial intelligence;
- observability;
- scalability;
- testing;
- production deployment.

ADGSTUDIOS DisputeGuard

Secure. Explainable. Auditable. Scalable.